Diamante Technical Coaching

Taming the Dragon: Software Craftsmanship for Coding Agents

Steven Diamante

Technical Coach & AI Consultant

Founder of Diamante Technical Coaching



This talk is for software engineers

Thinking about using coding agents

Using agents for side projects only

Using agents in production, but struggling

Using agents successfully, want to hear how others do it



Who am I?

- **Steven Diamante from Raleigh, NC**
  - Founder of Diamante Technical Coaching
  - AI Consultant at Artium AI
- **Avid XP practitioner**
  - pairing
  - TDD
  - refactoring
  - continuous integration
- **Over 2000 hours of Claude Code usage**
  - Building AI with AI in production
- **Hobbies**
  - Plays guitar and sings
  - Meditating for over 5 years
  - Chess and Poker
  - Travels the world with family



Agenda

- ❏ What are coding agents?
- ❏ Common Failure Modes
- ❏ What is Software Craftsmanship?
- ❏ Demo: Add a new feature to a Stock Tracker App
- ❏ Taming the Agentic Dragon Workflow

Diamante Technical Coaching

Get the Resources & Let's Connect



- 🔗 Connect on LinkedIn
- 👨‍🏫 Diamante Technical Coaching
- 💻 Slides/Code from this talk
- 🦾 Claude Code Skills & Agents
- 📧 AI Coding Newsletter
- 📅 Book a session

Coding Agents are here!

An agent runs tools in a loop to achieve a goal. Coding agents can read, modify, explore your codebase.











How can we partner with coding agents to deliver high-quality production code at a sustainable pace?



Common AI Coding Failure Modes for Software Engineers



One shot attempts

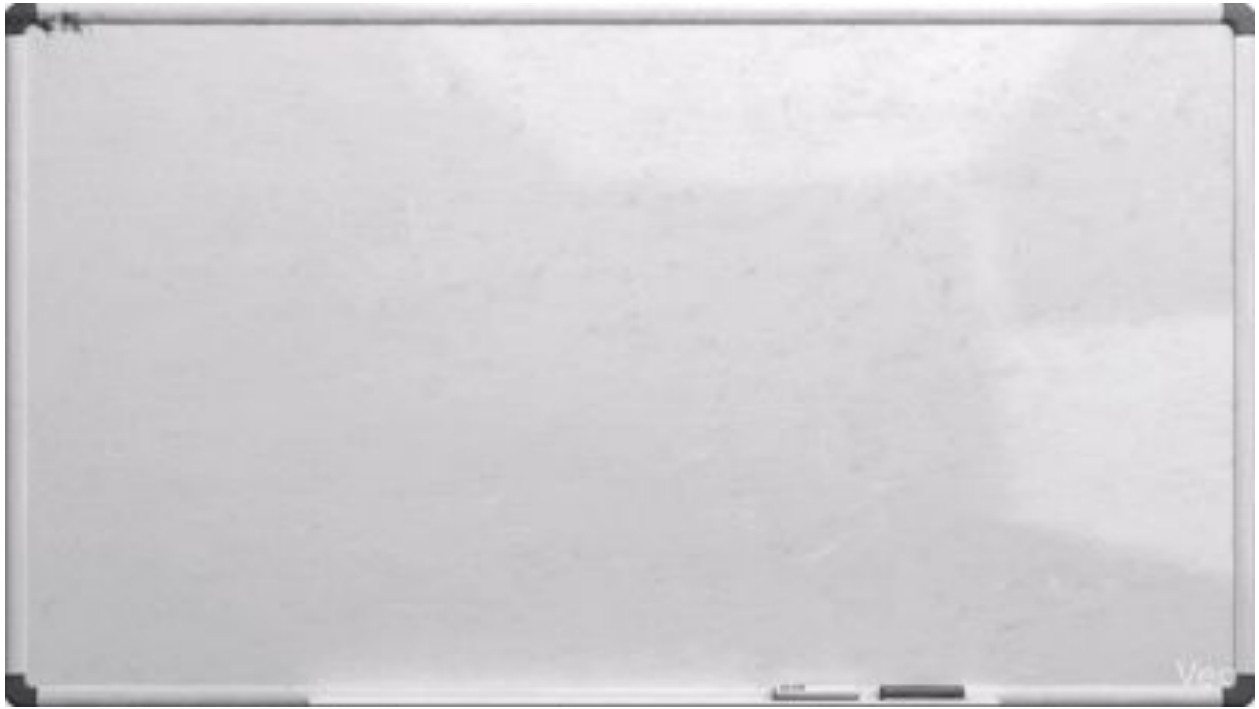



```
❯ Create a complete flappy bird clone. Don't stop until you're done.

  ~/p/flappy-bird-clome | Model: Sonnet 4.5 | Ctx: 0.0%
  (no git) | Cost: $0.00 | v2.1.5
  ⎇ no git | Style: default | Ctx: 0
  ▸▸ bypass permissions on (shift+tab to cycle)
```

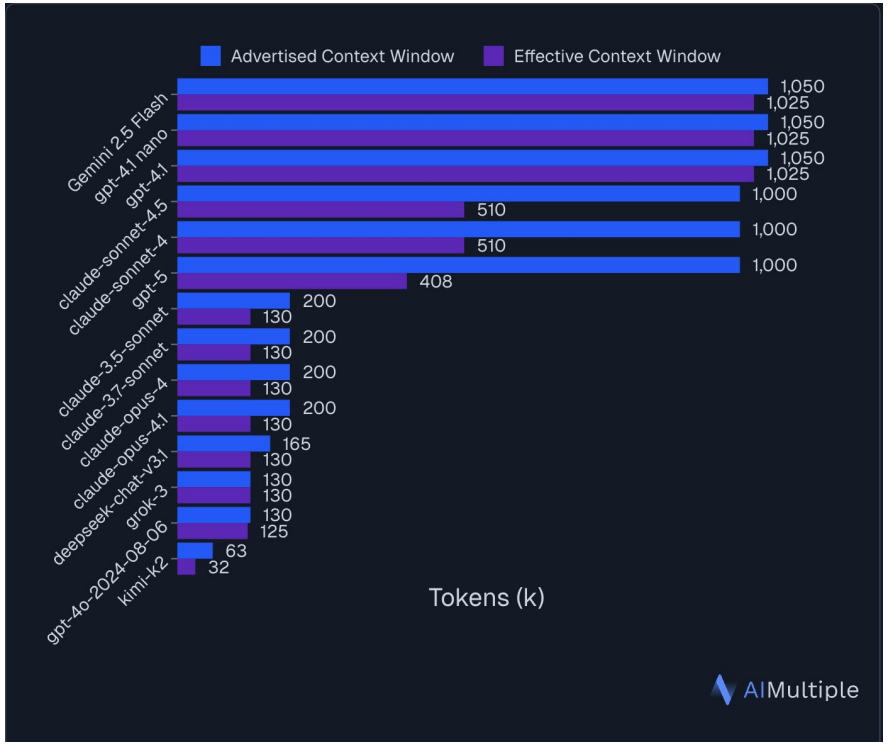
Failure Modes

Not respecting the context window



Failure Modes

Not respecting the context window



"Most models break much earlier than advertised. A model claiming 200k tokens typically becomes unreliable around **130k**, with **sudden performance drops** rather than gradual degradation."

— AIMultiple

Failure Modes

No ground rules



AGENTS.md

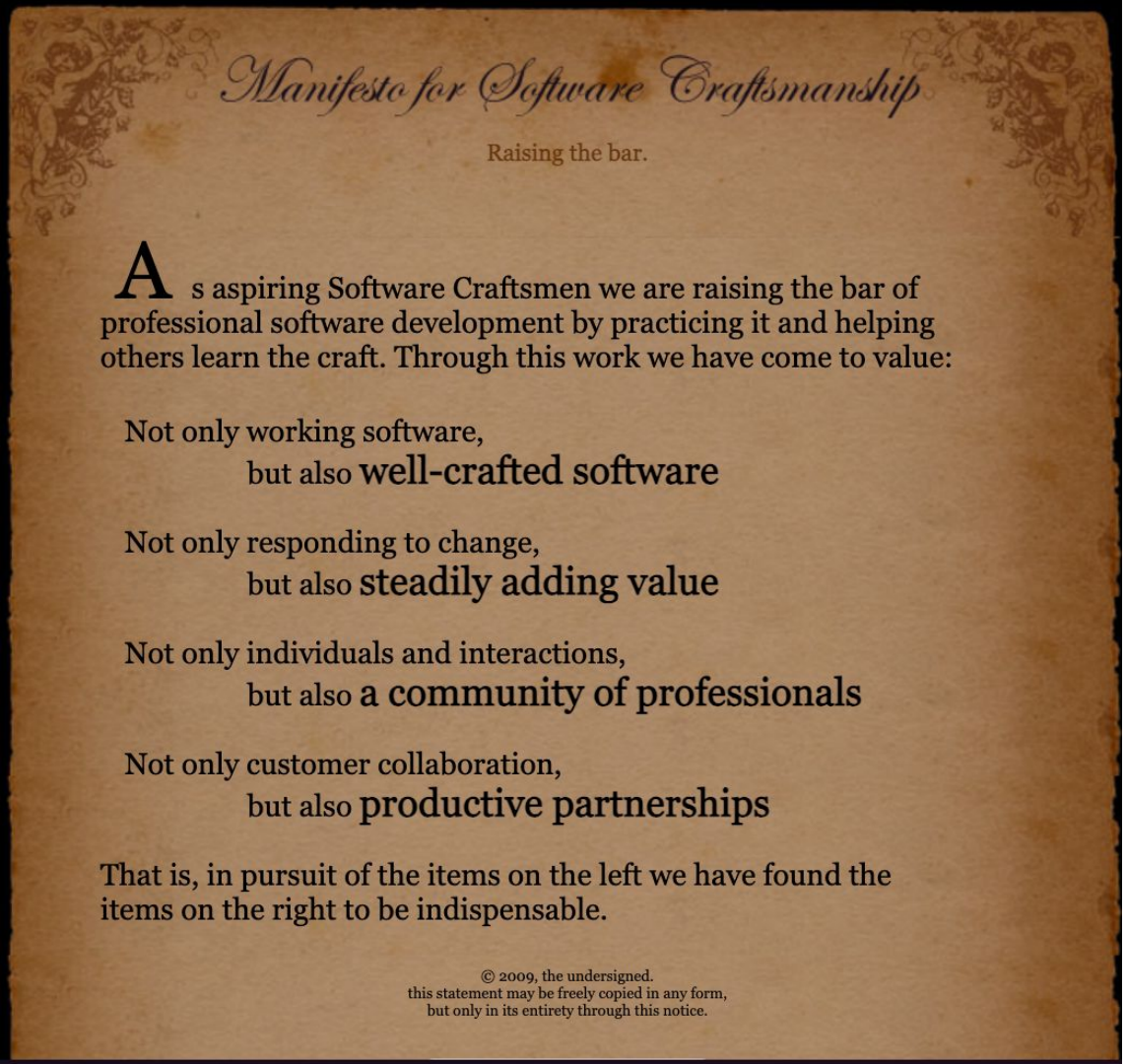
CLAUDE.md

Failure Modes

Telling the agent what to code



Failure Modes



Manifesto for Software Craftsmanship

Raising the bar.

As aspiring Software Craftsmen we are raising the bar of professional software development by practicing it and helping others learn the craft. Through this work we have come to value:

Not only working software,
but also **well-crafted software**

Not only responding to change,
but also **steadily adding value**

Not only individuals and interactions,
but also **a community of professionals**

Not only customer collaboration,
but also **productive partnerships**

That is, in pursuit of the items on the left we have found the items on the right to be indispensable.

© 2009, the undersigned.
this statement may be freely copied in any form,
but only in its entirety through this notice.

The Craftsmanship Comeback

- A movement to prioritize
  - quality
  - community
  - incremental business value
- XP practices like
  - TDD
  - Continuous Integration (TBD)
  - Refactoring
  - Social programming

Stock Tracker App

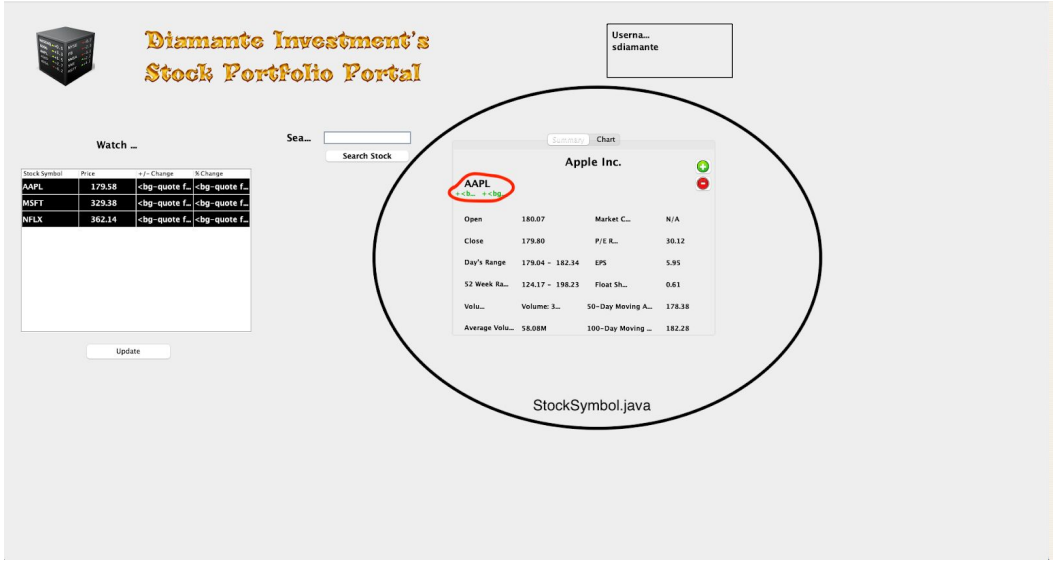
WHAT IT IS

- ❏ Modern web platform for real-time stock tracking
- ❏ Mobile-optimized for retail investors & traders
- ❏ Unified view of essential stock data

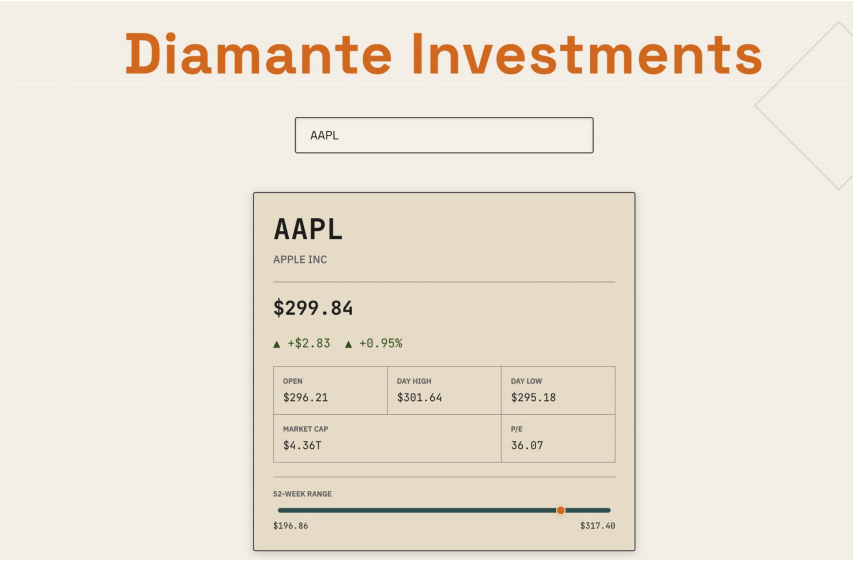
CORE CAPABILITIES

- ❏ Search & Discover Stocks
- ❏ Watchlist Management
- ❏ View interactive candlestick charts
- ❏ Comprehensive Metrics

Diamante Investments v1.0 -> v2.0

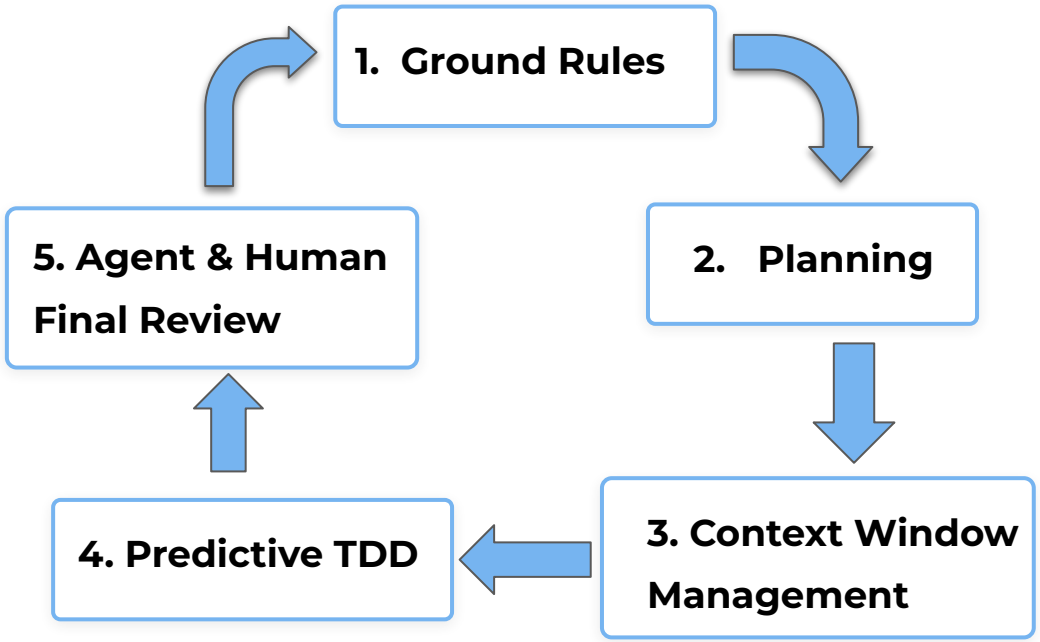


Java Swing Desktop Application built in 2017



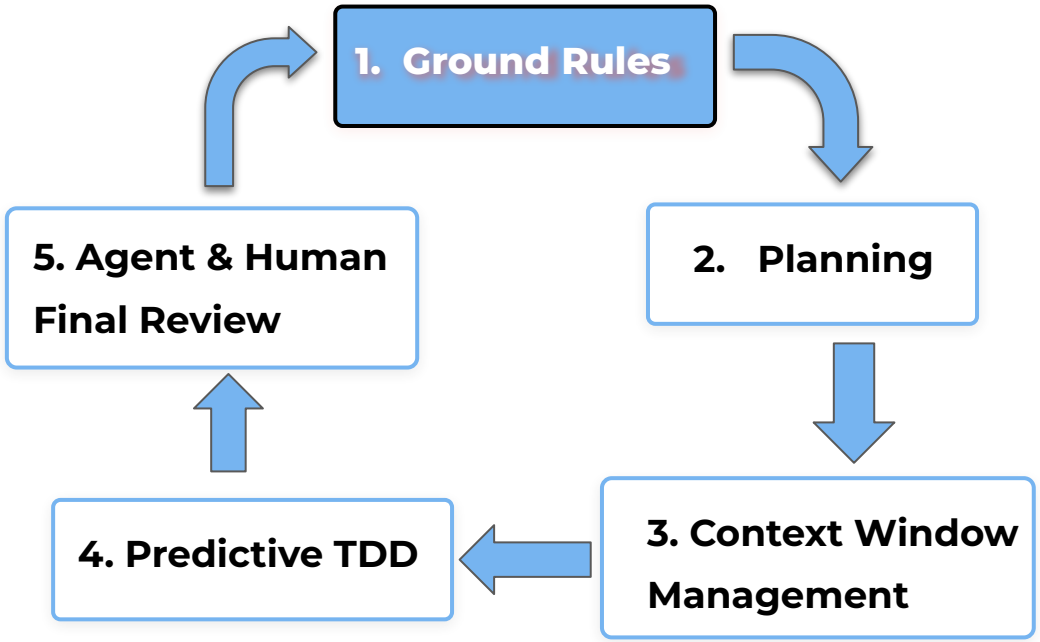
Current Web version

Taming the Agentic Dragon





Taming the Agentic Dragon





Ground Rules

Don’t repeat yourself in every chat

Give your agent rules and guidelines to follow

- ❏ CLAUDE.md, AGENTS.md files
- ❏ Working agreement with agent

Ask agent to help create rules

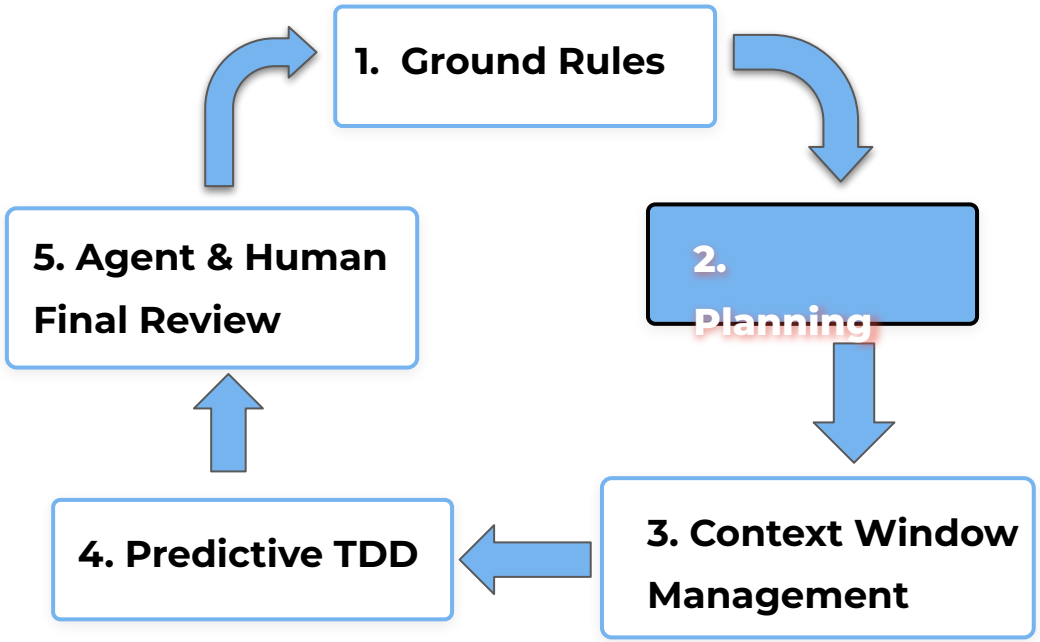
When working with an agent's defaults, they may not suit your needs. As you find yourself teaching and lecturing the agent, make a note to enforce rules.



Agent Skills

- Open Standard originally introduced in October 2025
  - Self-contained folder with instructions plus optional scripts/, references/, and assets/
- Progressive Disclosure - only loads skill metadata
  - Improved context management over slash commands
- Team sharable and cross-tool compatability

Taming the Agentic Dragon





Define Success Before You Code



- ❏ Gherkin User Stories
- ❏ **Given/When/Then** scenarios
- ❏ Clear acceptance criteria



Expected Outcomes

What **success** looks like

Verifiable results

Provide Context



Why are we doing this?

What problem does it solve?

Business value alignment



Integration Information

- ❏ API documentation
- ❏ System diagrams
- ❏ Existing code patterns

Interview Mode



❓

Ask Clarifying Questions

Let agent probe for details

Uncover edge cases

3️⃣

Give Me 3 Options

Different implementation approaches

Trade-off analysis

📐

For UI: Ask for ASCII Wireframes

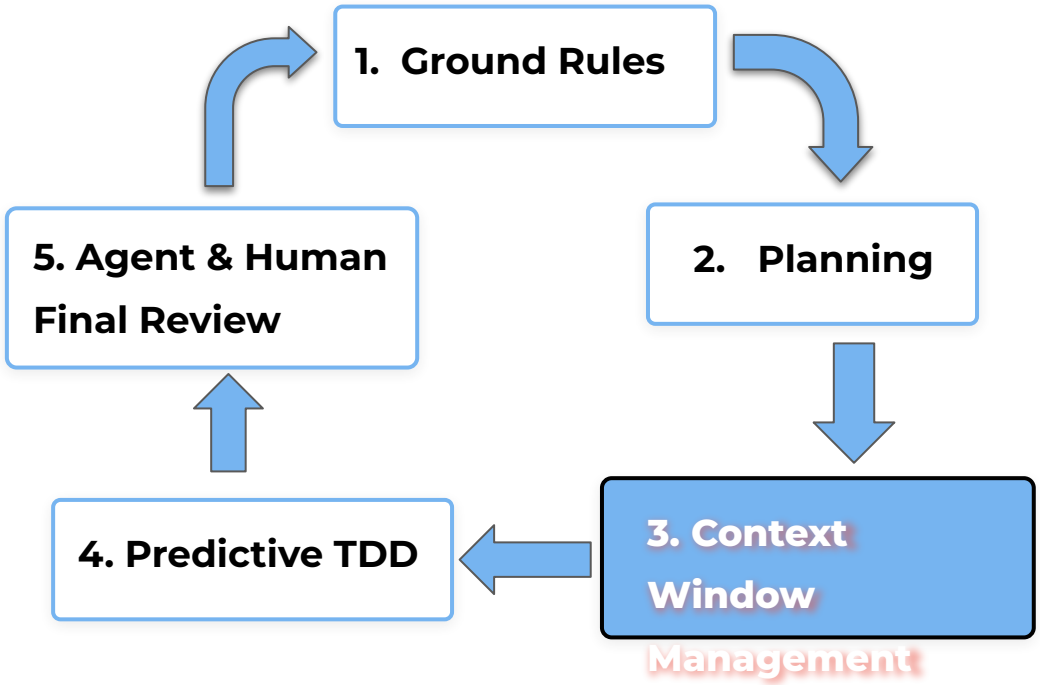
Visual layout planning before implementation

Review & Refine

⚠️ **WARNING: Read Every Line**

- 🐛 There's something wrong in the plan
- 🔧 Find it and correct it
- ❓ Challenge assumptions
- ✅ Verify completeness
- 🦾 Have another agent review the plan

Taming the Agentic Dragon





Diamante Technical Coaching

Context Window Management

Plan ready? Check your context window!

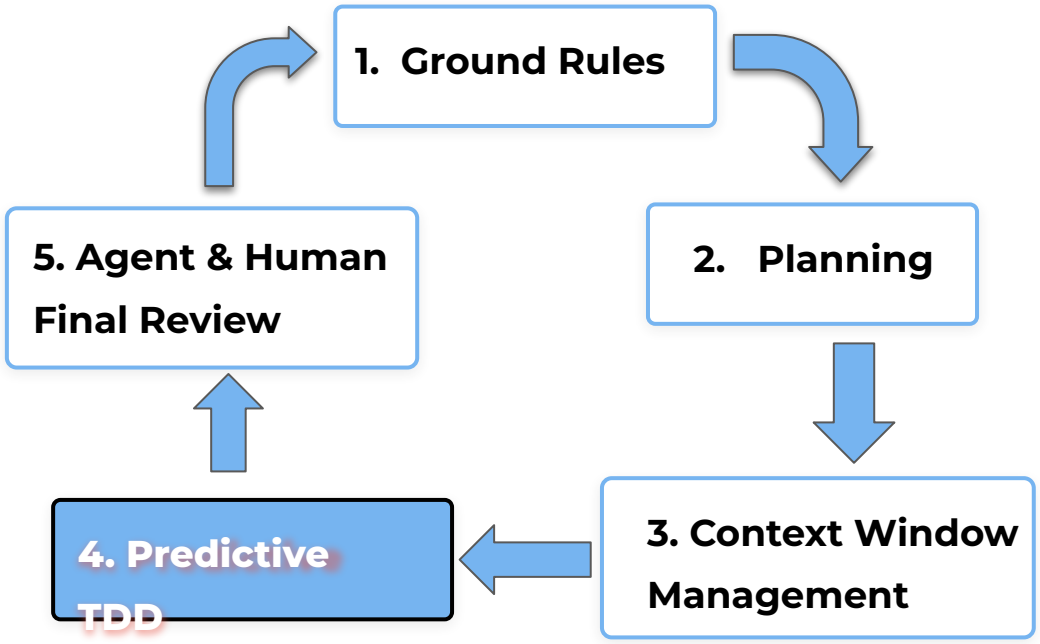
Models degrade at ~60% of context window

Extract plan to .md file?

OR

Continue with execution in same session?

Taming the Agentic Dragon

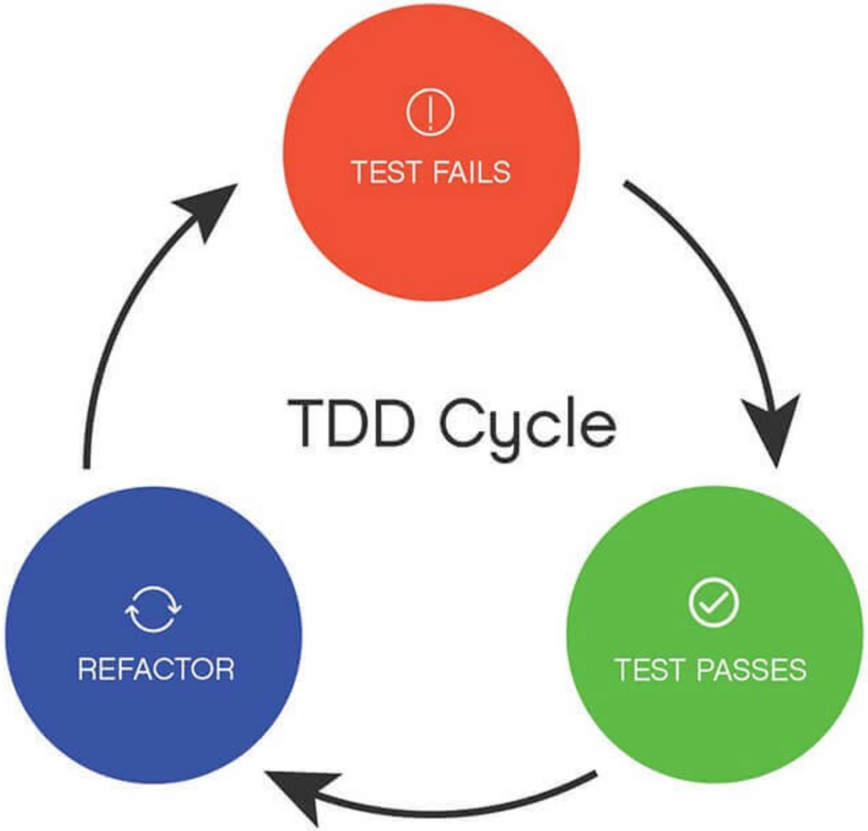




Predictive TDD

Ted M. Young (@JitterTed)

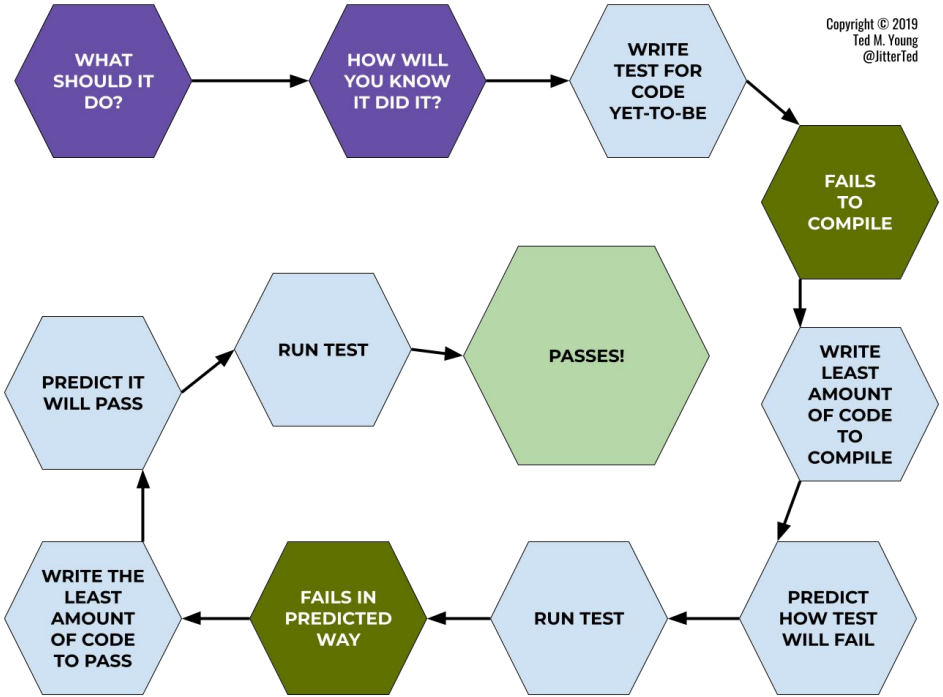
- ❏ TDD agent skill
- ❏ Test behaviors, not implementation
- ❏ Keep mocking minimal
- ❏ Prefer acceptance tests with test containers & mock servers
- ❏ Tight feedback loop for agent to self-correct



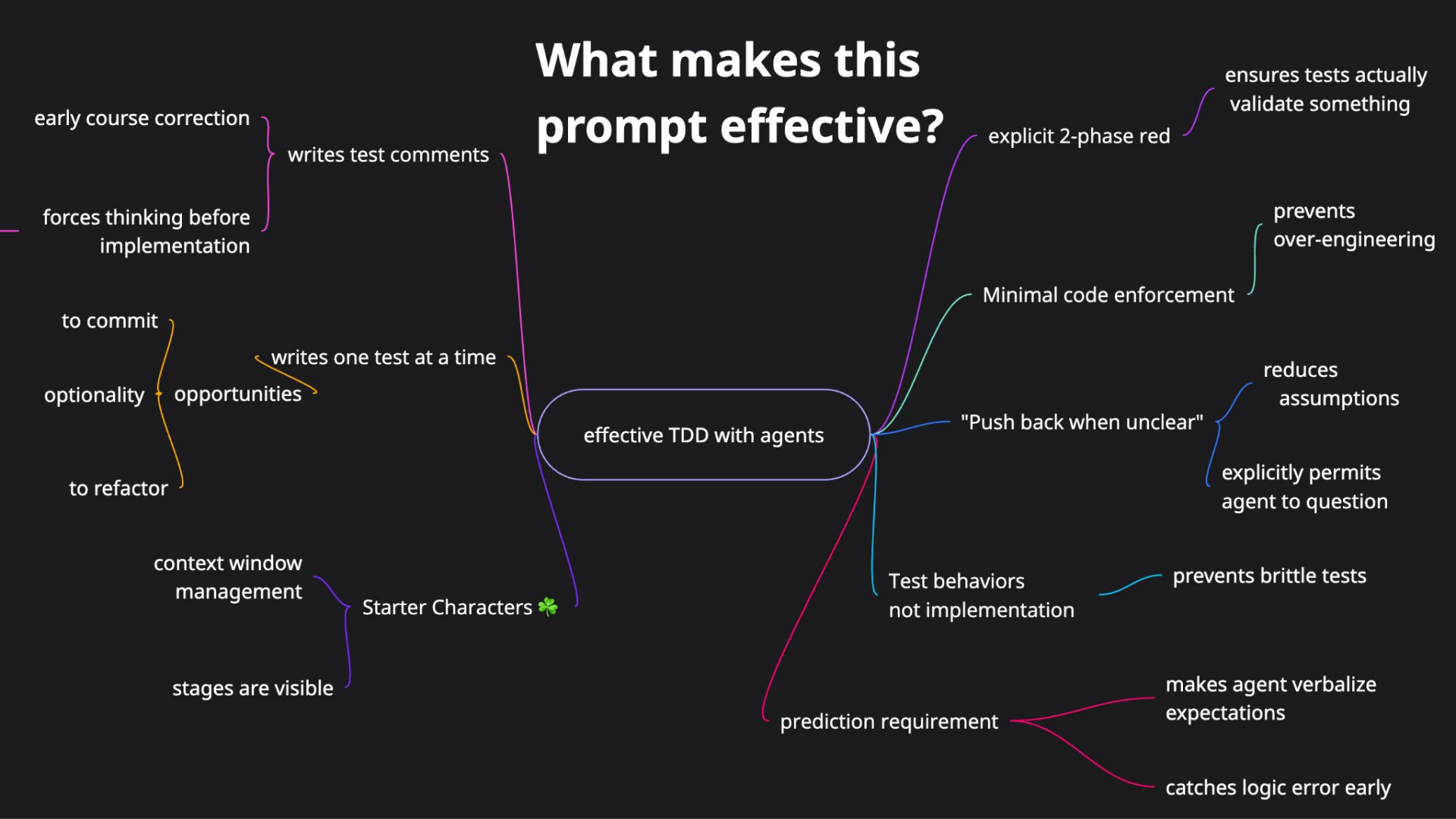
Predictive TDD

Ted M. Young (@JitterTed)





What makes this prompt effective?



How to Enforce Refactoring Step

Git hooks + strict linting rules

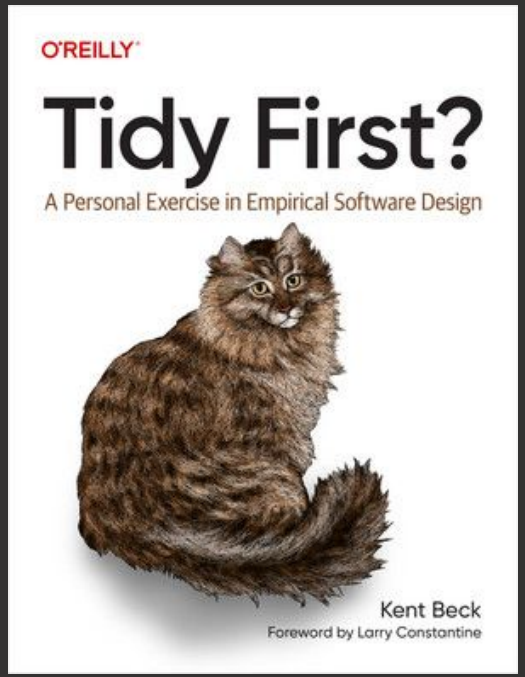
- ❑ precommit => build & format & lint & deduplicate
- ❑ prepush => run all tests

💡 Commit with additional coding agent to save unnecessary context

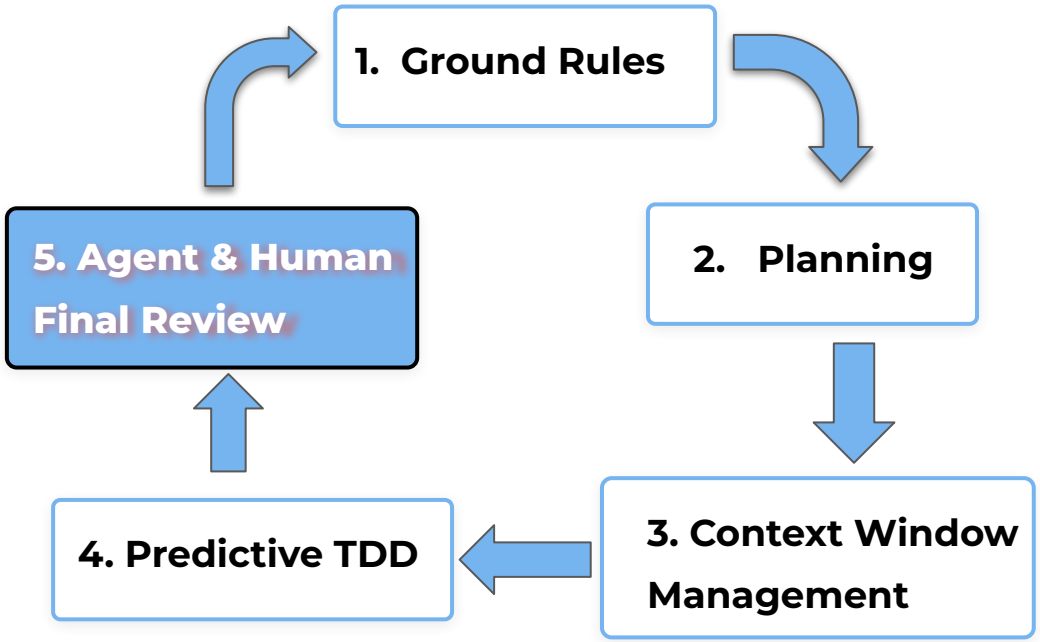
Tidy After?

Tidy Up Skill

- ❑ Micro refactors covered by tests
- ❑ Microcommits using Arlo's Commit Notation (e.g. - r <refactoring>)
- ❑ If tests fail, revert
- ❑ Remove dead code, improve names, extract methods, remove duplication, etc.



Taming the Agentic Dragon





Agent & Human Final Review



Manual Testing

Verify functionality works as expected



Playwright MCP

UI testing by agent

Automated exploratory testing



Fresh Agent Review

Code review with criteria

Unbiased perspective

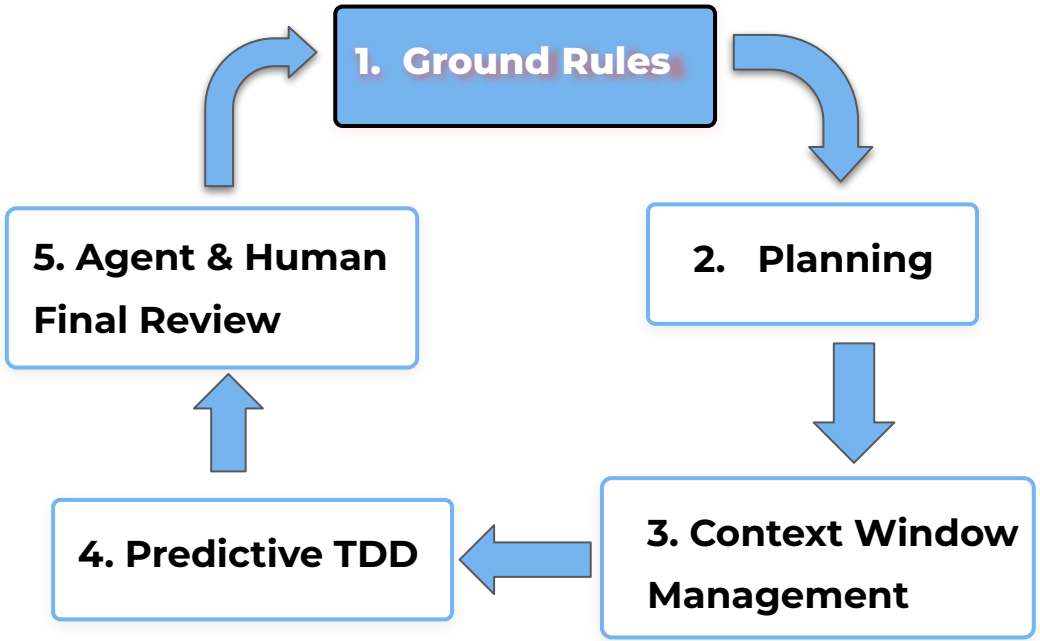


Human Final Review

Human oversight

Refactor as needed

Taming the Agentic Dragon



Remember

“I'm about to clear this conversation. Compile a list of the 5 most important things that would be useful to add to the CLAUDE.md. Tell me first for approval”

How can we partner with coding agents to deliver high-quality production code at a sustainable pace?



Takeaways

1. Create ground rules, stop repeating yourself
2. Plan first: Use interview mode, get 3 options, verify with ASCII wireframes
3. Watch your context window - extract plans at ~60%
4. Use Predictive TDD to catch errors early and enable self-correction
5. Multi-layer review: manual testing → agent review → human oversight
6. Experiment and find what works for you



Diamante Technical Coaching

Tame your Dragon Today!

Get the Resources & Let's Connect



🔗 Connect on LinkedIn
👨🏽‍💼 Diamante Technical Coaching
💻 Slides/Code from this talk
🦾 Claude Code Skills & Agents
📧 AI Coding Newsletter
📆 Book a session